Rules of use: what to share and what you own

Objectives

- Sort project material into share, redact first, or never share before it reaches a prompt
- Redact the secret and personal parts of a message while keeping the part that helps

Teaching

Start with a guess

Your script fails, and the error output includes your access token. You want an assistant's help.

All files, values and messages here are synthetic and fake. Nothing calls a real AI model.

Three lanes at the prompt boundary

Before anything goes into a prompt, put it in a lane. Share: code and docs with no secrets or personal data. Redact first: useful material with a secret or personal detail inside. Never share: material that is secret or personal as a whole, such as private keys or customer records.

Treat anything you paste as having left your control.

Redact, don't discard

Redacting keeps what helps and replaces what harms with a placeholder such as <TOKEN>. Error codes and file and line numbers usually help; tokens, passwords and personal details do not.

You also own the result. Submit code an assistant proposed, and you answer for it as if you wrote it. Use only tools your program or employer approves.

Apply it to a new message

A teammate drafts a prompt for help with a database connection. Same lanes, new material.

Model

The prompt boundary

Not sorted yet

- start.sh
- error output
- ~/.ssh/id_ed25519
- README.md
- test log
- customers.csv

Share

Redact first

Never share

What would be pasted

```
401 Unauthorized user=ana.lee@example.test token=FAKE-DEMO-7f3a9c at
start.sh line 6
```

Recap

- Sort before you paste: share, redact first, or never share.
- Redact the secret, keep the useful part: replace values with placeholders such as <TOKEN>.
- Treat anything you paste as out of your control.
- You answer for AI-proposed code you submit, and you use only approved tools.

Next topic: How Agile teams work: Scrum and user stories

Cheat sheet

Share	project code and docs with no secrets or personal data
Redact first	errors and logs with a token, password or email inside
Never share	private keys, password files, customer or personal records
Placeholder	NAME=<NAME>: keep the name, replace the value

Sources

Claims on this page were checked against the live official pages below on 6 October 2026 (UTC). Links open outside this lesson.

- OWASP LLM02:2025 Sensitive Information Disclosure: User education: avoid inputting sensitive information. Vendor-neutral guidance.
- Generating a new SSH key (GitHub Docs): ~/.ssh/id_ed25519 key files. The file without .pub is the private key.

Not tied to one source: the three-lane sort and placeholder style are this lesson's teaching model. Data handling and retention differ by provider and plan; follow your program or employer's approved-tool policy.

Static interaction alternatives

predict

Prediction (not scored). An error message you want help with contains an access token. Options: paste the whole error; paste it with the token replaced by a placeholder; describe it in one sentence with no error text; never ask about errors. Safest useful choice: paste the error with the token replaced by a placeholder. Pasting it whole sends the token out of your control; a one-sentence description is safe but leaves the assistant guessing; never asking is overcautious.

drag

Sort six items into three lanes. Share: start.sh (project script with no secrets) and README.md (how to run the project). Redact first: the error output (useful, but contains a token) and a test log (useful, but contains a teammate's email address). Never share: ~/.ssh/id_ed25519 (a private SSH key, secret as a whole) and customers.csv (real customer names and emails; write made-up sample rows instead).

explore

Redact an error line made of four parts: 401 Unauthorized, user=ana.lee@example.test, token=FAKE-DEMO-7f3a9c and at start.sh line 6. Redact the email (user=<EMAIL>) and the token (token=<TOKEN>). Keep 401 Unauthorized, which says what failed, and at start.sh line 6, which says where.

mcq

Graded check with a confidence rating. A draft pastes DB_HOST=db.example.test and a real-looking DB_PASSWORD. The revision that follows the rules keeps DB_HOST and the error and replaces the

password value with <DB_PASSWORD>. Sending it as written adds risk and no help, because the assistant proposes text and never connects. Removing everything leaves it guessing. Sending and then deleting the chat still exposes the secret. Retry, one line at a time: DB_PASSWORD= <DB_PASSWORD> is safe; a partly masked value still leaks most of the secret, the same value written as a sentence is not redacted, and the raw value is the secret itself.